

CSP 算法模板 • 打印合订本

语言：C++17（GCC）。这是一份本地离线资料，不需要联网。图默认 1 下标，字符串/网格默认 0 下标，各节另有说明。算法数值与中间运算须落在 `long long` 范围内。

快速选择

连通块、可达性 → 01 DFS / 03 BFS

枚举选择、排列、小规模搜索 → 02 回溯

最少步数、同时扩散、等权边 → 03 - 04 BFS

带权最短路，所有边权非负 → 05 Dijkstra

先后依赖、有向无环图最长路 → 06 拓扑排序

合并集合、是否连通 → 07 并查集

最小代价连接所有点 → 08 最小生成树

最大化最小值 / 最小化最大值，判定单调 → 09 二分答案

单点修改区间求和 → 10 树状数组

右侧第一个更大、滑动窗口极值 → 11 - 12 单调栈/队列

选物品、限制容量 → 13 背包

最长上升子序列 → 14 LIS

子串比较 / 精确匹配 → 15 哈希 / 16 KMP

大指数取模、质数模逆元 → 17 快速幂

区间修改区间和 → 附录线段树

使用方法

每节代码粘贴到下面公共头之后、`main` 之前。06 依赖 05 的 `Graph` 定义，08 依赖 07 的 `DSU`。纸质资料中的所有模板均已展开；最后附独立的 `Dijkstra` 和 `线段树` 完整程序。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = (1LL << 62);
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    // 读入 → 建图/初始化 → 调用 → 按题目要求输出
}
```

01 DFS：连通块遍历

来源：暑期训练营代码/Day10/图的遍历.cpp（题型参考）；通用迭代版补充

图下标 $1..n$ ； $O(n+m)$ 。用栈避免链状图递归爆栈。遍历标记不撤销；求连通块时从每个未访问点启动。

```
void dfs(int s, const vector<vector<int>>& g, vector<int>& vis) {
    stack<int> st; st.push(s); vis[s] = 1;
    while (!st.empty()) {
        int u = st.top(); st.pop();
        // 在这里处理 u
        for (int v : g[u]) if (!vis[v]) {
            vis[v] = 1; st.push(v);
        }
    }
}
// vector<int> vis(n+1); int cnt=0;
// for(int i=1;i<=n;++i) if(!vis[i]) { ++cnt; dfs(i,g,vis); }
```

02 DFS：回溯枚举

来源：暑期训练营代码/Day2/单词接龙.cpp

提炼“选择→递归→撤销”。下面枚举 $1..n$ 的全排列， $O(n*n!)$ （含输出）。题目限制使用两次时，把 used 改为计数；剪枝必须保证不丢解。

```
void permutations(int n) {
    vector<int> path, used(n+1);
    function<void()> dfs = [&]() {
        if ((int)path.size() == n) {
            for (int x : path) cout << x << ' ';
            cout << '\n'; return;
        }
        for (int x=1; x<=n; ++x) if (!used[x]) {
            used[x]=1; path.push_back(x);
            dfs();
            path.pop_back(); used[x]=0;
        }
    };
    dfs();
}
```

03 BFS：无权图最短路

来源：暑期训练营代码/Day2/血色先锋队.cpp（从网格推广到图）

边权相同才可用普通 BFS 求最短路； $O(n+m)$ 。入队时标记，-1 表示不可达。多个源点同时赋 0 入队就是多源 BFS。

```
vector<int> bfs(int s, const vector<vector<int>>& g) {
    vector<int> d(g.size(), -1); queue<int> q;
    d[s]=0; q.push(s);
    while (!q.empty()) {
        int u=q.front(); q.pop();
        for (int v:g[u]) if(d[v]==-1) {
            d[v]=d[u]+1; q.push(v);
        }
    }
    return d;
}
```

04 BFS：网格、多源扩散

来源：暑期训练营代码/Day2/血色先锋队.cpp

网格下标 $0..n-1, 0..m-1$ ，# 为障碍；源点坐标必须有效。 $O(nm + \text{源点数})$ 。只有一个源点时传 $\{\{sx, sy\}\}$ 。返回 -1 表示不可达或障碍。

```
vector<vector<int>> grid_bfs(const vector<string>& a,
                           const vector<pair<int, int>>& src) {
    int n=a.size(), m=n ? a[0].size() : 0;
    vector<vector<int>> d(n, vector<int>(m, -1));
    queue<pair<int, int>> q;
    for(auto [x, y]:src) if(a[x][y]!='#' && d[x][y]==-1) {
        d[x][y]=0; q.push({x, y});
    }
    int dx[]={1, -1, 0, 0}, dy[]={0, 0, 1, -1};
    while(!q.empty()) {
        auto [x, y]=q.front(); q.pop();
        for(int k=0; k<4; ++k) {
            int nx=x+dx[k], ny=y+dy[k];
            if(nx<0 || nx>=n || ny<0 || ny>=m) continue;
            if(a[nx][ny]=='#' || d[nx][ny]!=-1) continue;
            d[nx][ny]=d[x][y]+1; q.push({nx, ny});
        }
    }
    return d;
}
```

05 Dijkstra：非负边权最短路

来源：日常学习/单源最短路径.cpp

所有边权必须非负。图下标 $1..n$ ；堆优化 $O((n+m)\log(n+m))$ ，含平行边。距离改用 long long；所有有效距离须小于 INF。无向边加两次。

```
using Graph = vector<vector<pair<int, ll>>>;
vector<ll> dijkstra(int s, const Graph& g) {
    vector<ll> d(g.size(), INF);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    d[s]=0; pq.push({0, s}); // 堆中先距离、后编号
    while(!pq.empty()) {
        auto [du, u]=pq.top(); pq.pop();
        if(du!=d[u]) continue;
        for(auto [v, w]:g[u]) if(w < INF-du && du+w<d[v]) {
            d[v]=du+w; pq.push({d[v], v});
        }
    }
    return d;
}
// Graph g(n+1); g[u].push_back({v, w});
// auto d=dijkstra(s, g); if(d[t]==INF) ...; // 按题意输出不可达
// 需要路径：松弛成功时 pre[v]=u，再从终点沿 pre 回溯并 reverse。
```

06 拓扑排序与 DAG 最长路

来源：日常学习/拓扑排序.cpp；日常学习/最长路BFS.cpp

仅用于有向图。拓扑序长度小于 n 表示有环。DAG 最长路按拓扑序转移，允许负边；不可达状态不转移。 $O(n+m)$ 。

```
vector<int> topo(const Graph& g) {
    int n=(int)g.size()-1; vector<int> deg(n+1),order; queue<int> q;
    for(int u=1;u<=n;++u) for(auto [v,w]:g[u]) ++deg[v];
    for(int u=1;u<=n;++u) if(!deg[u]) q.push(u);
    while(!q.empty()) {
        int u=q.front();q.pop();order.push_back(u);
        for(auto [v,w]:g[u]) if(--deg[v]==0) q.push(v);
    }
    return order;
}

pair<bool,vector<ll>> dag_longest(int s,const Graph& g) {
    auto order=topo(g); vector<ll> d(g.size(),-INF);
    if(order.size()+1!=g.size()) return {false,d};
    d[s]=0;
    for(int u:order) if(d[u]!=-INF)
        for(auto [v,w]:g[u]) d[v]=max(d[v],d[u]+w);
    return {true,d}; // false:有环; -INF:不可达; 路径和须在(-INF, INF)内
}
```

07 并查集

来源：日常学习/最小生成树.cpp；暑期训练营代码/Day5/并查集.cpp

动态合并连通块；路径压缩+按大小合并，均摊 $O(\alpha(n))$ 。DSU $uf(n)$ ； $uf.unite(u,v)$ ；

$uf.find(u)==uf.find(v)$ 。

```
struct DSU {
    vector<int> fa,sz;
    DSU(int n):fa(n+1),sz(n+1,1) { iota(fa.begin(),fa.end(),0); }
    int find(int x) { return fa[x]==x ? x : fa[x]=find(fa[x]); }
    bool unite(int x,int y) {
        x=find(x);y=find(y); if(x==y) return false;
        if(sz[x]<sz[y]) swap(x,y);
        fa[y]=x;sz[x]+=sz[y];return true;
    }
};
```

08 Kruskal：最小生成树

来源：日常学习/最小生成树.cpp

无向图，允许负边和平行边， $n \geq 1$ 。依赖并查集。 $O(m \log m)$ 。独立返回是否连通，避免 -1 与合法负权答案混淆。

```
struct Edge { int u,v; ll w; };
pair<bool,ll> kruskal(int n,vector<Edge> e) {
    sort(e.begin(),e.end(),[](const Edge& a,const Edge& b){return a.w<b.w;});
    DSU uf(n); ll ans=0; int cnt=0;
    for(auto x:e) if(uf.unite(x.u,x.v)) {
        ans+=x.w; if(++cnt==n-1) break;
    }
    return {cnt==n-1,ans};
}
```

09 二分答案

来源：暑期训练营代码/Day1/跳石头.cpp

原题用逐个枚举答案，这里提炼为二分。check 必须单调，区间内至少有一个可行值；每次 $O(\log R)$ 次 check。范围非负且 $hi-lo+1$ 不溢出。

```
// true true ... false false: 找最后一个 true (最大可行值)
template<class Check> ll last_true(ll lo, ll hi, Check check) {
    while(lo < hi) {
        ll mid = lo + (hi - lo) / 2;
        if(check(mid)) lo = mid; else hi = mid - 1;
    }
    return lo;
}
// false false ... true true: 找第一个 true (最小可行值)
template<class Check> ll first_true(ll lo, ll hi, Check check) {
    while(lo < hi) {
        ll mid = lo + (hi - lo) / 2;
        if(check(mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
}
// 跳石头: pos 是升序内部石头位置, 最后追加终点 L。
// auto check = [&](ll gap) { int removed = 0; ll last = 0;
//     for(ll x : pos) { if(x - last < gap) ++removed; else last = x; }
//     return removed <= M; };
// cout << last_true(0, L, check);
// 最后一段过短计时数等价于移除前一个保留石头, 终点仍保留。
```

10 树状数组：单点加、区间和

来源：暑期训练营代码/Day5/树状数组1.cpp

严格使用 $1..n$ 下标， $\text{add}(0, \dots)$ 会死循环！修改、查询 $O(\log n)$ 。BIT $b(n)$ ；逐点 $b.\text{add}(i, a[i])$ 建树。

```
struct BIT {
    int n; vector<ll> t;
    BIT(int n) : n(n), t(n+1) {}
    void add(int x, ll v) { for(; x <= n; x += x & -x) t[x] += v; }
    ll sum(int x) const { ll s = 0; for(; x > 0; x -= x & -x) s += t[x]; return s; }
    ll query(int l, int r) const { return sum(r) - sum(l-1); }
};
// 区间加、单点查：用差分建 BIT, add(l, v), r < n 时 add(r+1, -v), sum(x)。
```

11 单调栈：右侧第一个更大元素

来源：暑期训练营代码/Day5/单调栈.cpp

数组下标 $0..n-1$ ；答案为下标，无答案为 -1 。 $O(n)$ 。严格更大用 $<$ ，大于等于用 $<=$ 。

```
vector<int> next_greater(const vector<ll>& a) {
    vector<int> ans(a.size(), -1); stack<int> st;
    for(int i = 0; i < (int)a.size(); ++i) {
        while(!st.empty() && a[st.top()] < a[i]) {
            ans[st.top()] = i; st.pop();
        }
        st.push(i);
    }
    return ans;
}
```

12 单调队列：滑动窗口最小值

来源：暑期训练营代码/Day5/滑动窗口.cpp

$1 \leq k \leq n$, $O(n)$ 。队列存下标；改 $>=$ 为 $<=$ 就是窗口最大值。

```
vector<ll> window_min(const vector<ll>& a, int k) {
    deque<int> q; vector<ll> ans;
    for(int i=0; i<(int)a.size(); ++i) {
        while(!q.empty() && q.front() <= i-k) q.pop_front();
        while(!q.empty() && a[q.back()] >= a[i]) q.pop_back();
        q.push_back(i);
        if(i >= k-1) ans.push_back(a[q.front()]);
    }
    return ans;
}
```

13 背包：01 与完全

来源：日常学习/装箱问题.cpp；暑期训练营代码/Day3/疯狂的采药.cpp

$O(nV)$ ，空间 $O(V)$ 。 v 为正体积， w 为价值；这里表示“容量至多 V ”，允许不选。恰好装满：除 $dp[0]=0$ 外设 $-INF$ ，转移前判可达。

```
ll knapsack(int V, const vector<int>& v, const vector<ll>& w, bool complete) {
    vector<ll> dp(V+1, 0);
    for(int i=0; i<(int)v.size(); ++i) {
        if(complete) { // 完全背包：每种无限次，正序
            for(int j=v[i]; j<=V; ++j) dp[j] = max(dp[j], dp[j-v[i]]+w[i]);
        } else { // 01背包：每种一次，倒序
            for(int j=V; j>=v[i]; --j) dp[j] = max(dp[j], dp[j-v[i]]+w[i]);
        }
    }
    return dp[V];
}
// 装箱最小剩余容量：价值等于体积，答案 V-knapsack(V, v, w, false)。
```

14 LIS：最长严格上升子序列

来源：暑期训练营代码/Day3/最长上升子序列.cpp

$O(n \log n)$ 。 $lower_bound$ 求严格上升， $upper_bound$ 求不下降。 $tails$ 本身不保证是原数组的一条子序列。

```
int lis(const vector<ll>& a) {
    vector<ll> tails;
    for(ll x:a) {
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if(it == tails.end()) tails.push_back(x); else *it = x;
    }
    return tails.size();
}
```

15 字符串哈希：子串比较

来源：暑期训练营代码/Day9/字符串匹配.cpp

修正前缀下标。get(l,r) 表示 0 下标半开区间 [l,r)，O(1)。unsigned long long 自然溢出；哈希有碰撞风险，相等不构成严格证明，精确匹配可用下一节 KMP。

```
struct StringHash {
    using ull=unsigned long long;
    vector<ull> h,p;
    StringHash(const string& s):h(s.size()+1),p(s.size()+1,1) {
        for(int i=0;i<(int)s.size();++i) {
            h[i+1]=h[i]*131+(unsigned char)s[i]+1;
            p[i+1]=p[i]*131;
        }
    }
    ull get(int l,int r) const { return h[r]-h[l]*p[r-l]; }
};
```

16 KMP：精确字符串匹配（补充）

来源：补充模板，非原代码直接提取

O(n+m)。返回每次匹配的 0 下标起点，支持重叠匹配；此函数约定空模式返回空列表。

```
vector<int> kmp(const string& s,const string& p) {
    int m=p.size(); if(!m) return {};
    vector<int> pi(m),ans;
    for(int i=1,j=0;i<m;++i) {
        while(j && p[i]!=p[j]) j=pi[j-1];
        if(p[i]==p[j]) ++j;
        pi[i]=j;
    }
    for(int i=0,j=0;i<(int)s.size();++i) {
        while(j && s[i]!=p[j]) j=pi[j-1];
        if(s[i]==p[j]) ++j;
        if(j==m) {ans.push_back(i-m+1);j=pi[j-1];}
    }
    return ans;
}
```

17 快速幂与逆元

来源：暑期训练营代码/Day8/模意义下的乘法逆元.cpp（逆元主题，补充快速幂写法）

O(log b)。b>=0，1<=mod<=2³¹-1，保证 long long 乘法不溢出。费马逆元仅适用于 p 为质数且 a%p!=0；不是任意模数都能用。

```
ll mod_pow(ll a,ll b,ll mod) {
    a=(a%mod+mod)%mod;ll ans=1%mod;
    while(b) {if(b&1) ans=ans*a%mod;a=a*a%mod;b>>=1;}
    return ans;
}
// 质数模数 p 下逆元：mod_pow(a,p-2,p)。
// 批量逆元 (1<=n<p, p 为质数)：
// vector<ll> inv(n+1);inv[1]=1;
// for(int i=2;i<=n;++i) inv[i]=(p-p/i)*inv[p/i]%p;
```

附录 A：前缀和、差分与 STL（补充）

```
// 以下是独立片段，变量按题目定义；a 和 pre 均分配 n+1，1 下标。
vector<ll> pre(n+1);
for(int i=1;i<=n;++i) pre[i]=pre[i-1]+a[i];
```

```

ll answer=pre[r]-pre[l-1]; // 闭区间 [l,r]

// 差分: d 分配 n+2; 初值 d[i]=a[i]-a[i-1], 令 a[0]=0。
d[1]+=v;d[r+1]-=v;
for(int i=1;i<=n;++i) a[i]=a[i-1]+d[i];

// 二维前缀和, 数组四周预留一圈 0:
s[i][j]=a[i][j]+s[i-1][j]+s[i][j-1]-s[i-1][j-1];
// [x1..x2] × [y1..y2] 的和:
ll ans=s[x2][y2]-s[x1-1][y2]-s[x2][y1-1]+s[x1-1][y1-1];

sort(a.begin(),a.end());
a.erase(unique(a.begin(),a.end()),a.end()); // 排序去重, 也可用于离散化
int pos=lower_bound(a.begin(),a.end(),x)-a.begin(); // 第一个 >=x; 可能等于size
// upper_bound: 第一个 >x。使用前数组必须升序!
priority_queue<int,vector<int>,greater<int>> pq; // 小根堆
// getline 前若刚使用 cin >>, 用 getline(cin >> ws,line) 可跳过后导空白;
// 如果空行/行首空格有意义, 则改用 cin.ignore(...) 丢弃上一行剩余内容。

```

你原代码里需要特别注意的地方

1. 血色先锋队.cpp: 右侧边界用 `graph.size()` 比较列号, 非正方形网格会出错; 整理版分别用 `n`、`m`。
2. 单源最短路径.cpp: `int` 距离可能溢出, 原 `INF` 也可能比合法路径短; 整理版改为 `long long`。不可达输出值必须看题面, 不能所有题都输出 2147483647。
3. 最长路BFS.cpp: 反复入队更新不是普通 BFS 的最短步数性质; 可达正环可能导致不停更新。DAG 用拓扑 DP, 一般有环图不能直接套本模板求最长简单路径。
4. 字符串匹配.cpp: 从 `i=0` 调用 `get` 时访问 `pos-1`, 下标越界; 整理版用 `n+1` 前缀数组和半开区间。
5. 跳石头.cpp: 原程序逐个尝试距离, 答案大时效率不足; 保留贪心判定, 换二分答案。
6. 装箱问题.cpp: 原状态按物品数建数组, 不能直接当作标准 01 背包; 应按容量定义状态。此处仅指未带 02 的这个文件。
7. 最小生成树.cpp: 用 `-1` 表示失败会和负权合法答案混淆; 整理版用 `bool` 单独表示连通性。

提交前 30 秒检查

- 数据范围能否承受复杂度? $n \approx 10^5$ 时一般先考虑 $O(n \log n)$ 或 $O(n)$, 不要直接双重枚举。
- 图是有向还是无向? 边是否非负? 点编号是否从 0 开始? 是否有重边、自环、不连通?
- 多组数据清空容器与状态; `long long` 接住求和和乘法 (`1LL*a*b`)。
- BFS 入队时标记; 回溯返回要撤销; 01 背包倒序、完全背包正序。
- 数组越界、空容器、不可达值、负数答案、`n=1`, 以及输出格式。
- 递归深度可能达到 `n` 时优先改迭代; 不要把短样例能运行等同于大数据安全。

这份资料是选取常用主题后的整理, 不是对全部历史题解的正确性审计。补充内容已标明。

附录 B: Dijkstra 完整程序

输入: $n\ m\ s$, 接着 m 行有向边 $u\ v\ w\ (w \geq 0)$ 。输出源点到各点距离, 不可达输出 -1 ; 按实际题意修改。无向图需加反向边。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = (1LL << 62);
using Graph = vector<vector<pair<int, ll>>>
vector<ll> dijkstra(int s, const Graph& g) {
    vector<ll> d(g.size(), INF);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    d[s]=0; pq.push({0, s}); // 堆中先距离、后编号
    while(!pq.empty()) {
        auto [du, u]=pq.top(); pq.pop();
        if(du!=d[u]) continue;
        for(auto [v, w]:g[u]) if(w < INF-du && du+w<d[v]) {
            d[v]=du+w; pq.push({d[v], v});
        }
    }
    return d;
}
// Graph g(n+1); g[u].push_back({v, w});
// auto d=dijkstra(s, g); if(d[t]==INF) ...; // 按题意输出不可达
// 需要路径: 松弛成功时 pre[v]=u, 再从终点沿 pre 回溯并 reverse。

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m, s; cin >> n >> m >> s;
    Graph g(n+1);
    for(int i=0; i<m; ++i) {int u, v; ll w; cin >> u >> v >> w; g[u].push_back({v, w});}
    auto d=dijkstra(s, g);
    for(int i=1; i<=n; ++i) cout << (d[i]==INF ? -1 : d[i]) << (i==n ? '\n' : ' ');
}
```

附录 C：线段树完整程序

输入 n m 、 n 个数；1 1 r k ：区间加 k ；2 1 r ：区间求和。1 下标， $n \leq 100000$ 。多组数据须清空 `lazy`。

```
#include <iostream>
#include <vector>
using namespace std;

typedef long long ll;

const int MAXN = 100005;

ll a[MAXN];          // 原始数组
ll tree[MAXN << 2];  // 线段树，区间和
ll lazy[MAXN << 2];  // 懒标记

// 建树
void build(int node, int l, int r) {
    if (l == r) {
        tree[node] = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(node << 1, l, mid);
    build(node << 1 | 1, mid + 1, r);
    tree[node] = tree[node << 1] + tree[node << 1 | 1];
}

// 下传懒标记
void push_down(int node, int l, int r) {
    if (lazy[node] == 0) return;
    int mid = (l + r) >> 1;
    int left = node << 1;
    int right = node << 1 | 1;
    // 更新左子节点
    tree[left] += lazy[node] * (mid - l + 1);
    lazy[left] += lazy[node];
    // 更新右子节点
    tree[right] += lazy[node] * (r - mid);
    lazy[right] += lazy[node];
    // 清除当前节点懒标记
    lazy[node] = 0;
}

// 区间更新 [ql, qr] += val
void update(int node, int l, int r, int ql, int qr, ll val) {
    if (ql <= l && r <= qr) {
        tree[node] += val * (r - l + 1);
        lazy[node] += val;
        return;
    }
    push_down(node, l, r);
    int mid = (l + r) >> 1;
    if (ql <= mid) update(node << 1, l, mid, ql, qr, val);
    if (qr > mid) update(node << 1 | 1, mid + 1, r, ql, qr, val);
    tree[node] = tree[node << 1] + tree[node << 1 | 1];
}

// 区间查询 [ql, qr] 的和
ll query(int node, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) {
        return tree[node];
    }
```

```
    }
    push_down(node, l, r);
    int mid = (l + r) >> 1;
    ll ans = 0;
    if (ql <= mid) ans += query(node << 1, l, mid, ql, qr);
    if (qr > mid) ans += query(node << 1 | 1, mid + 1, r, ql, qr);
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
    }
    build(1, 1, n);

    while (m--) {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1) {
            ll k;
            cin >> k;
            update(1, 1, n, x, y, k);
        } else {
            cout << query(1, 1, n, x, y) << '\n';
        }
    }
    return 0;
}
```